

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

SAINT JEAN UNIVERSITY INSTITUTE

INSTITUT UNIVERSITAIRE SAINT JEAN



REPUBLIC OF CAMEROON

Peace-Work-Fatherland

SAINT JEAN INGENIEUR

SAINT JEAN INGENIEUR



THEORIE DES SYSTEMES D'EXPLOITATION

REALISATION DE LA COMMANDE PS

OPTION : SYSTEME RESEAUX & TELECOMMUNICATION

INGE 3 SRT

Membres du groupe

- ONGUENE ANNE ALIMBA
- SEUMO TCHATCHOUANG Bertrand
- TAMBO DJOUONTZO STEPHEN DARREN
- NGANKAK DZUKOU LOUIS-GEORGES
- FOTSO ERYANGE VERDIANE

Sous la supervision de :

Monsieur NGUIMBUS Emmanuel

Année académique :2025/2026

SOMMAIRE

SOMMAIRE	2
INTRODUCTION GENERALE	3
PRESENTATION DE PS	4
LIEN AVEC LA THEORIE DES SYSTEMES D'EXPLOITATION	5
ANALYSE DU FONCTIONNEMENT DE /PROC	6
CAHIER DE CHARGES FONCTIONNEL	9
CAHIER DES CHARGES TECHNIQUE	11
REPARTITION DES FONCTIONNALITES ENTRE MEMBRES	12
ARCHITECTURE LOGICIELLE DU PROGRAMME	13
STRUCTURE DE DONNEES UTILISEES.....	14
DESCRIPTION DETAILLEE DES PRINCIPALES FONCTIONS.....	15
ALGORITHMES PRINCIPAUX.....	19
GESTION DES ERREURS ET ROBUSTESSE.....	22
INTEGRATION AVEC LINUX ET POSIX	23
COMPILATION ET EXECUTION	24
TESTS ET VALIDATION	24
ORGANISATION COLLABORATIVE DU PROJET	25
DIFFICULTES RENCONTREES ET SOLUTIONS APPORTEES.....	26
LIMITES ACTUELLES DU PROGRAMME	27
PERSPECTIVES D'AMELIORATION	29
CONCLUSION GENERALE.....	30

INTRODUCTION GENERALE

La gestion des processus constitue l'un des aspects fondamentaux des systèmes d'exploitation modernes. Sous Linux, chaque programme en cours d'exécution est représenté par un processus, identifié par un PID, c'est-à-dire un identifiant unique attribué par le noyau. L'administrateur système, le développeur ou l'utilisateur avancé doit régulièrement consulter l'état de ces processus afin de surveiller l'activité de la machine, diagnostiquer des erreurs, identifier des processus gourmands en ressources ou comprendre les relations parent-enfant entre programmes.

Dans ce contexte, la commande `ps`, qui signifie **Process Status**, occupe une place centrale. Elle permet d'afficher un instantané des processus actifs du système à un moment donné. Elle fournit des informations telles que le PID, le terminal associé, le temps CPU consommé, le nom de la commande, l'utilisateur propriétaire du processus, le processus parent, la mémoire utilisée, les groupes de processus ou encore les threads.

Notre présent projet consiste à concevoir et implémenter une version personnalisée de la commande `ps`, appelée `myps`, en langage C sous Linux. L'objectif n'est pas de reproduire toute la complexité de la commande originale, mais de développer un outil fonctionnel capable de lire les informations des processus depuis le système de fichiers virtuel `/proc`, de les filtrer, de les trier et de les afficher selon plusieurs options.

Notre travail s'inscrit donc directement dans le cadre du cours de Théorie des Systèmes d'Exploitation. Il permet de mettre en pratique plusieurs notions importantes : processus, threads, hiérarchie des processus, espace utilisateur, espace noyau, fichiers virtuels, appels système, gestion de la mémoire dynamique et programmation système en C.

PRESENTATION DE PS

La commande `ps` est un outil standard des systèmes Unix et Linux. Elle permet d'afficher les processus en cours d'exécution sur le système. Contrairement à une commande comme `top`, qui affiche les processus en temps réel, `ps` fournit une photographie instantanée de l'état du système au moment de son exécution.

Par défaut, la commande `ps` affiche généralement les processus associés au terminal courant. Elle présente des informations de base comme le PID, le terminal, le temps CPU et la commande lancée. Cependant, son comportement peut être largement modifié grâce à des options. Ces options permettent d'afficher les processus de tous les utilisateurs, d'inclure les processus sans terminal, de filtrer par PID, par nom, par utilisateur, de modifier les colonnes affichées ou encore de représenter la hiérarchie des processus.

Dans le cadre de notre projet, nous avons développé une commande `myps` inspirée de `ps`. Elle permet notamment d'afficher les processus du système, de filtrer les résultats selon plusieurs critères, d'adapter le format d'affichage, de trier les processus et de visualiser certaines informations avancées comme les threads ou le contexte de sécurité SELinux.

L'objectif principal de cette commande est donc de comprendre comment un outil système récupère les informations internes du noyau Linux, puis les transforme en données lisibles par l'utilisateur.

LIEN AVEC LA THEORIE DES SYSTEMES

D'EXPLOITATION

Ce projet est fortement lié aux notions étudiées dans le cours de Théorie des Systèmes d'Exploitation. En effet, la commande `myps` manipule directement des informations relatives aux processus Linux.

Un processus est une instance d'un programme en cours d'exécution. Chaque processus possède un identifiant appelé PID. Il peut également posséder un processus parent, identifié par le PPID. Ces notions sont essentielles pour comprendre la hiérarchie des processus, notamment le fait qu'un processus peut créer d'autres processus.

Le projet met également en évidence la distinction entre l'espace utilisateur et l'espace noyau. Le programme `myps` s'exécute en espace utilisateur, mais il accède à des informations maintenues par le noyau Linux grâce au **système de fichiers virtuel `/proc`**. Ce dernier sert d'interface entre l'utilisateur et le noyau.

La notion de thread intervient également dans ce projet. Un thread peut être vu comme une unité d'exécution légère appartenant à un processus. L'option permettant d'afficher les threads oblige le programme à parcourir le dossier **`/proc/[pid]/task`**, qui contient les différents threads associés à un processus.

Enfin, le projet met en pratique plusieurs aspects techniques de la programmation système : ouverture et lecture de fichiers, parcours de répertoires, gestion des erreurs, allocation dynamique de mémoire, conversion d'identifiants numériques en noms d'utilisateurs, tri de structures et formatage de l'affichage.

ANALYSE DU FONCTIONNEMENT DE /PROC

1. Présentation générale de /proc

Sous Linux, /proc est un système de fichiers virtuel. Cela signifie qu'il ne correspond pas à un dossier classique stocké physiquement sur le disque dur. Il est créé et maintenu dynamiquement par le noyau. Les fichiers qu'il contient permettent d'accéder à des informations système en temps réel.

Chaque processus actif possède un dossier dans /proc. Ce dossier porte comme nom le PID du processus. Par exemple, le dossier /proc/1234 contient les informations relatives au processus dont le PID est 1234.

Pour développer notre commande myps, nous avons donc parcouru le dossier /proc, identifié les entrées numériques correspondant aux PID, puis lu plusieurs fichiers internes afin de récupérer les informations nécessaires à l'affichage.

2. Le fichier /proc/[pid]/stat

Le fichier **stat** contient des informations compactes sur un processus. Toutes les informations sont placées sur une seule ligne. Ce fichier est moins lisible pour un humain, mais il est très utile pour un programme, car les champs sont organisés selon un ordre précis.

Dans notre projet, ce fichier est utilisé pour récupérer plusieurs informations essentielles, notamment le PID, le PPID, l'état du processus, le nom court de la commande, le groupe de processus, la session, la mémoire résidente et les temps CPU consommés en mode utilisateur et en mode noyau.

Ces informations permettent ensuite de calculer certains champs affichés à l'écran, comme le temps CPU total ou le pourcentage approximatif d'utilisation du processeur.

3. Le fichier `/proc/[pid]/status`

Le fichier `status` contient des informations présentées sous forme de paires clé-valeur. Il est beaucoup plus lisible que le fichier `stat`. Dans notre programme, il permet notamment de récupérer l'UID, le GID et le nombre de threads du processus.

L'UID permet d'identifier l'utilisateur propriétaire du processus. À partir de cet identifiant numérique, le programme peut récupérer le nom d'utilisateur correspondant grâce à la fonction `getpwuid`.

Le GID permet quant à lui d'identifier le groupe réel du processus. Cette information est utile pour les options de filtrage par groupe.

4. Le fichier `/proc/[pid]/cmdline`

Le fichier **`cmdline`** contient la ligne de commande complète utilisée pour lancer le processus. Contrairement à une chaîne de caractères classique, les arguments y sont séparés par des caractères nuls. Pour rendre l'affichage lisible, le programme remplace ces caractères nuls par des espaces.

Ce fichier est utilisé pour afficher la colonne **`CMD`** ou **`COMMAND`**. Lorsque ce fichier est vide, cela peut correspondre à un thread noyau ou à un processus particulier. Dans ce cas, notre programme prévoit un affichage adapté, par exemple **`[kernel thread]`**.

5. Le fichier `/proc/[pid]/attr/current`

Le fichier `attr/current` peut être utilisé pour récupérer le contexte de sécurité SELinux associé à un processus. Cette information intervient dans l'option `-Z`, qui permet d'afficher une colonne de sécurité.

Tous les systèmes Linux ne disposent pas forcément de SELinux activé. Pour cette raison, lorsque le contexte n'est pas disponible, le programme affiche une valeur par défaut comme ?.

CAHIER DE CHARGES FONCTIONNEL

Le cahier des charges fonctionnel présente les fonctionnalités que doit offrir la commande `myps` du point de vue de l'utilisateur.

1. Affichage par défaut

La commande **myps**, lorsqu'elle est exécutée sans option, doit afficher une liste de processus avec des colonnes de base. Ces colonnes permettent d'identifier rapidement chaque processus, son terminal associé, le temps CPU consommé et la commande correspondante.

L'affichage par défaut doit rester clair, simple et proche de l'esprit de la commande **ps**.

2. Option de sélection et de filtrage

Le programme doit permettre de filtrer les processus affichés. Ces options sont importantes, car un système peut contenir un très grand nombre de processus. Sans filtrage, l'affichage peut devenir difficile à lire.

Les options de sélection permettent notamment :

- d'afficher les processus de tous les utilisateurs ;
- d'inclure les processus sans terminal ;
- de sélectionner un processus précis à partir de son PID ;
- de filtrer les processus selon le terminal associé ;
- de filtrer les processus selon leur nom ;
- de filtrer les processus selon leur groupe réel.

3. Options de formatage

Le programme doit également permettre de modifier les colonnes affichées. Selon le besoin, l'utilisateur peut vouloir un affichage simple ou un affichage plus détaillé.

Les options de formatage permettent par exemple :

- d'afficher le nom de l'utilisateur propriétaire du processus ;
- d'afficher le PID du processus parent ;
- d'afficher la mémoire utilisée ;
- d'afficher le pourcentage CPU ;
- d'afficher le groupe de processus ;
- d'afficher la session ;
- d'utiliser le nom court de la commande plutôt que la ligne de commande complète.

Ces options rendent la commande plus flexible et plus utile pour l'analyse système.

4. Options de visualisation avancée

Certaines options permettent d'aller au-delà d'un simple tableau de processus. Le programme doit pouvoir afficher une représentation hiérarchique des processus afin de montrer les relations parent-enfant. Il doit aussi pouvoir afficher les threads associés aux processus.

Le programme doit également permettre le tri des résultats selon différents critères, comme le PID, le PPID, le nom, la mémoire ou le pourcentage CPU. Cette fonctionnalité facilite l'analyse lorsque le nombre de processus affichés est important.

Enfin, l’affichage du contexte SELinux apporte une information supplémentaire dans les environnements où la sécurité renforcée est activée.

CAHIER DES CHARGES TECHNIQUE

Le cahier des charges technique présente les contraintes et choix techniques retenus pour réaliser le projet.

Le programme est développé en langage C, car ce langage permet une interaction directe avec les fichiers système et les interfaces POSIX. Il est particulièrement adapté à la programmation système sous Linux.

Le programme doit être exécuté sur un système Linux disposant du système de fichiers virtuel /proc. Il n’est donc pas conçu pour fonctionner directement sous Windows, sauf dans un environnement compatible comme WSL.

La compilation peut être réalisée avec gcc. Le programme doit utiliser uniquement des bibliothèques standards ou disponibles sur un système Linux classique.

Le programme doit respecter plusieurs contraintes :

- parcourir correctement le dossier /proc ;
- distinguer les dossiers de processus des autres fichiers système ;
- lire les fichiers stat, status, cmdline et éventuellement attr/current ;
- gérer les erreurs d’ouverture ou de lecture ;
- supporter les processus qui disparaissent pendant l’exécution ;
- stocker dynamiquement les informations des processus ;
- appliquer les filtres demandés ;
- trier les processus lorsque nécessaire ;

- afficher les résultats de manière lisible.

La robustesse est un point essentiel, car le contenu de /proc change en permanence. Un processus peut apparaître ou disparaître pendant l'exécution de myps. Le programme doit donc éviter les arrêts brutaux en cas de fichier inaccessible.

REPARTITION DES FONCTIONNALITES ENTRE MEMBRES

Le projet a été organisé de manière collaborative afin de répartir les fonctionnalités entre les membres du groupe. La répartition a été pensée pour éviter les doublons et garantir que chaque membre contribue à un type précis de fonctionnalité.

Les fonctionnalités ont été regroupées en plusieurs catégories :

- options de sélection ;
- options de formatage ;
- options de visualisation ;
- options de hiérarchie ;
- options liées aux threads ;
- options de sécurité ;
- options de tri.

Chaque membre du groupe a travaillé sur un ensemble d'options spécifiques. Cette organisation a facilité le développement parallèle des fonctionnalités, puis leur intégration dans une version finale commune du programme.

Après le développement individuel, les différentes contributions ont été regroupées dans le programme principal. Cette étape d'intégration a nécessité une harmonisation du style de code, des noms de variables, des structures utilisées et des formats d'affichage.

ARCHITECTURE LOGICIELLE DU PROGRAMME

L'architecture logicielle du programme myps repose sur une organisation logique en plusieurs blocs fonctionnels. Même si le code peut être contenu dans un seul fichier source, son fonctionnement peut être compris comme un ensemble de modules.

Le premier module est chargé de l'analyse des arguments de la ligne de commande. Il détermine les options activées par l'utilisateur et initialise les variables correspondantes.

Le deuxième module est chargé du parcours de /proc. Il ouvre le répertoire /proc, lit ses entrées, puis conserve uniquement celles dont le nom est numérique. Ces entrées correspondent aux PID des processus actifs.

Le troisième module est chargé de la lecture des informations des processus. Pour chaque PID, le programme lit les fichiers nécessaires afin de remplir une structure représentant un processus.

Le quatrième module est chargé du filtrage. Il vérifie si le processus courant respecte les critères demandés par l'utilisateur. Si ce n'est pas le cas, le processus n'est pas conservé pour l'affichage.

Le cinquième module est chargé du stockage dynamique. Les processus retenus sont stockés dans un tableau dynamique, dont la taille est augmentée au fur et à mesure lorsque cela est nécessaire.

Le sixième module est chargé du tri. Lorsque l'utilisateur demande un tri particulier, le tableau des processus est trié selon le critère choisi.

Le septième module est chargé de l'affichage. Il affiche d'abord l'en-tête correspondant aux options choisies, puis chaque ligne de processus avec les colonnes appropriées.

Enfin, un module spécifique gère l'affichage hiérarchique et un autre gère l'affichage des threads.

Le fonctionnement général peut donc être résumé ainsi :

1. analyser les arguments ;
2. ouvrir /proc ;
3. parcourir les entrées numériques ;
4. lire les informations des processus ;
5. appliquer les filtres ;
6. stocker les processus valides ;
7. trier les résultats ;
8. afficher les informations ;
9. libérer la mémoire utilisée.

STRUCTURE DE DONNEES UTILISEES

La structure principale utilisée dans le programme est la structure Processus. Elle permet de regrouper toutes les informations utiles sur un processus ou un thread. Cette structure contient d'abord des champs d'identification comme le PID, le PPID, l'UID et le GID. Le PID identifie le processus, le PPID identifie son parent, l'UID identifie l'utilisateur propriétaire et le GID identifie le groupe réel.

Elle contient ensuite des informations d'état comme le nom du processus, son état, son terminal associé et sa ligne de commande complète. Le nom du processus provient généralement du fichier stat, tandis que la ligne de commande complète provient du fichier **cmdline**.

La structure contient aussi des informations liées aux ressources, notamment la mémoire résidente, le pourcentage CPU et le temps CPU total consommé par le processus. Pour les options avancées, elle contient également le groupe de processus, l'identifiant de session, le contexte SELinux, le TGID et le nombre de threads.

Cette structure joue un rôle central dans le programme. Elle permet de stocker les informations extraites de /proc sous une forme organisée, puis de les filtrer, les trier et les afficher facilement.

DESCRIPTION DETAILLEE DES PRINCIPALES FONCTIONS

1. La fonction `est_un_nombre()`

La fonction `est_un_nombre` vérifie si une chaîne de caractères contient uniquement des chiffres. Elle est utilisée pendant le parcours de /proc. En effet, le dossier /proc contient à la fois des dossiers numériques correspondant aux processus et d'autres fichiers ou dossiers système comme `cpuinfo`, `meminfo` ou `uptime`. Pour identifier les processus, le programme doit donc sélectionner uniquement les entrées dont le nom est numérique. Cette fonction retourne 1 si la chaîne est composée uniquement de chiffres, et 0 dans le cas contraire.

2. La fonction `lire_stat()`

La fonction **`lire_stat`** lit le fichier `/proc/[pid]/stat`. Elle extrait les informations principales du processus, notamment le PID, le PPID, l'état, le nom, le groupe de processus, la session, la mémoire résidente et les temps CPU. Elle est l'une des plus importantes du programme, car elle fournit la majorité des informations nécessaires à l'affichage. Elle calcule également le temps CPU total en additionnant le temps passé en mode utilisateur et le temps passé en mode noyau. Ces valeurs sont converties en secondes à l'aide de la fréquence d'horloge du système.

3. Fonctions `lire_uid` et `lire_gid`

Les fonctions `lire_uid` et `lire_gid` lisent le fichier `/proc/[pid]/status`. Elles permettent respectivement de récupérer l'identifiant utilisateur et l'identifiant de groupe du processus. Ces informations sont importantes pour l'affichage de l'utilisateur propriétaire du processus et pour le filtrage par groupe.

4. Fonction `lire_cmdline`

La fonction `lire_cmdline` lit le fichier `/proc/[pid]/cmdline`. Elle récupère la ligne de commande complète utilisée pour lancer le processus. Comme les arguments y sont séparés par des caractères nuls, la fonction remplace ces caractères par des espaces afin d'obtenir une sortie lisible. Lorsque le fichier est vide, la fonction affiche une valeur adaptée, par exemple **`[kernel thread]`**. Lorsque le fichier est inaccessible, elle peut afficher **`[defunct]`**.

5. Fonction **lire_tty**

La fonction **lire_tty** récupère le terminal associé à un processus. Elle lit les informations disponibles dans le fichier **status**. Si le processus ne possède pas de terminal associé, la fonction retourne `?`. Cela concerne souvent les démons, les services ou certains processus système.

6. Fonction **lire_contexte_Z**

La fonction **lire_contexte_Z** est utilisée lorsque l'option de sécurité est activée. Elle lit le fichier `/proc/[pid]/attr/current` pour récupérer le contexte SELinux du processus. Si le fichier n'existe pas ou n'est pas accessible, la fonction place une valeur par défaut. Cela permet au programme de continuer à fonctionner même lorsque SELinux n'est pas disponible.

7. Fonction **parse_args**

La fonction **parse_args** analyse les arguments fournis par l'utilisateur lors de l'exécution de la commande. Elle active les variables correspondant aux options détectées. Elle prend en charge les options simples, les options groupées et certaines options avec argument comme le PID, le terminal, le nom du processus ou le groupe. Cette fonction permet de séparer clairement la phase d'analyse des options du reste du programme.

8. Fonction **doit_afficher**

La fonction **doit_afficher** décide si un processus doit être conservé pour l'affichage. Elle applique les différents filtres activés par l'utilisateur. Par exemple, si l'utilisateur demande un PID précis, la fonction vérifie que le processus courant possède ce PID. Si l'utilisateur filtre par nom, la fonction vérifie que le nom du processus correspond au critère demandé. Cette fonction centralise la logique de filtrage, ce qui rend le programme plus lisible.

9. Fonction `comparer_processus`

La fonction **`comparer_processus`** est utilisée avec `qsort` pour trier le tableau des processus. Elle compare deux processus selon un critère choisi.

Les critères possibles peuvent être le PID, le PPID, la mémoire, le nom ou le pourcentage CPU. Si aucun critère particulier n'est demandé, le tri par défaut se fait généralement selon le PID.

10. Fonction `lister_threads`

La fonction `lister_threads` permet d'énumérer les threads d'un processus. Pour cela, elle parcourt le dossier `/proc/[pid]/task`. Chaque entrée numérique de ce dossier correspond à un TID, c'est-à-dire un identifiant de thread. Cette fonction est utilisée lorsque l'option d'affichage des threads est activée.

11. Fonctions `afficher_entete` et `afficher_ligne`

La fonction `afficher_entete` affiche les noms des colonnes en fonction des options choisies. Par exemple, un affichage simple ne contient pas les mêmes colonnes qu'un affichage détaillé. La fonction **`afficher_ligne`** affiche les informations d'un processus sur une ligne. Elle adapte le format de sortie selon les options activées.

12. Fonction `afficher_arbre`

La fonction `afficher_arbre` permet d'afficher les processus sous forme hiérarchique. Elle utilise les relations entre PID et PPID pour représenter les processus parents et leurs enfants. Il s'agit ainsi d'une approche récursive. Elle cherche les processus dont le PPID correspond au parent courant, les affiche avec une indentation, puis rappelle la fonction pour afficher leurs enfants.

13. Fonction main

La fonction **main** constitue le point d'entrée du programme. Elle orchestre toutes les étapes : analyse des arguments, ouverture de **/proc**, collecte des processus, filtrage, tri, affichage et libération de la mémoire. Elle assure donc la coordination entre toutes les autres fonctions du programme.

ALGORITHMES PRINCIPAUX

1. Algorithme général

L'algorithme général du programme commence par l'analyse des options saisies par l'utilisateur. Ensuite, le programme ouvre le répertoire `/proc` et parcourt toutes ses entrées.

Pour chaque entrée numérique, il considère qu'il s'agit d'un processus. Il lit alors les fichiers nécessaires pour récupérer les informations du processus. Ces informations sont stockées dans une structure.

Le programme vérifie ensuite si le processus respecte les filtres demandés. Si c'est le cas, il l'ajoute à un tableau dynamique.

Après le parcours de tous les processus, le tableau est trié. Enfin, le programme affiche les résultats selon le format demandé et libère la mémoire utilisée.

2. Algorithme de parcours de `/proc`

Le parcours de `/proc` se fait à l'aide des fonctions `opendir`, `readdir` et `closedir`.

L'algorithme peut être résumé ainsi :

- ouvrir le dossier `/proc` ;
- lire chaque entrée du dossier ;
- vérifier si le nom de l'entrée est numérique ;

- convertir ce nom en PID ;
- lire les informations du processus correspondant ;
- fermer le dossier à la fin du traitement.

Par cette méthode on peut obtenir la liste des processus actifs sans utiliser directement la commande ps.

3. Algorithme de filtrage

L'algorithme de filtrage consiste à appliquer une série de conditions sur chaque processus.

Si un filtre par PID est activé, le processus est conservé uniquement si son PID correspond au PID demandé. Si le filtre par utilisateur ou par groupe est activé, le programme compare l'UID ou le GID du processus avec la valeur attendue.

Si l'option permettant d'inclure les processus sans terminal n'est pas activée, les processus dont le terminal vaut ? peuvent être ignorés.

Le filtrage permet donc de réduire la liste des processus à afficher selon les besoins de l'utilisateur.

4. Algorithme de tri

Le tri des processus est effectué après leur collecte. Le programme utilise la fonction standard qsort, qui nécessite une fonction de comparaison. La fonction de comparaison observe le critère de tri choisi par l'utilisateur. Si le critère est pid, les processus sont comparés selon leur PID. Si le critère est cpu, ils sont comparés selon leur pourcentage CPU. Si le critère est nom, ils sont comparés alphabétiquement. En l'absence de critère de tri, le programme trie les processus par PID.

5. Algorithme d’affichage hiérarchique

L’affichage hiérarchique repose sur les relations parent-enfant entre processus. Chaque processus possède un PPID, qui correspond au PID de son processus parent. L’algorithme commence à partir d’un processus parent donné. Il cherche ensuite tous les processus dont le PPID correspond à ce parent. Pour chaque enfant trouvé, il l’affiche avec une indentation, puis recherche récursivement ses propres enfants.

6. Algorithme d’affichage des threads

Lorsque l’option d’affichage des threads est activée, le programme ne se limite plus aux processus. Pour chaque processus, il ouvre le dossier `/proc/[pid]/task`. Chaque entrée numérique de ce dossier représente un thread. Le programme lit alors les informations de chaque thread et les affiche avec des colonnes spécifiques, comme le TID, le TGID et le nombre de threads.

GESTION DES ERREURS ET ROBUSTESSE

La gestion des erreurs est un aspect essentiel de ce projet. En effet, le système de fichiers /proc est dynamique. Un processus peut exister au moment où son PID est lu, puis disparaître avant que le programme n'ait terminé de lire ses fichiers.

Le programme doit donc gérer les erreurs d'ouverture de fichiers. Si un fichier stat, status ou cmdline est inaccessible, le programme ne doit pas s'arrêter brutalement. Il doit ignorer le processus concerné ou afficher une valeur par défaut.

Le programme doit également gérer les erreurs d'arguments. Si l'utilisateur fournit une option inconnue, un PID invalide ou une option nécessitant un argument sans fournir cet argument, le programme doit signaler l'erreur.

La mémoire dynamique doit aussi être gérée correctement. Lorsque le tableau des processus devient trop petit, le programme utilise une réallocation. En cas d'échec de cette allocation, il doit libérer les ressources déjà utilisées avant de s'arrêter.

Enfin, certains processus peuvent ne pas avoir de terminal, de ligne de commande ou de contexte SELinux. Le programme prévoit donc des valeurs de remplacement afin de conserver un affichage cohérent.

INTEGRATION AVEC LINUX ET POSIX

Le programme myps s'appuie sur plusieurs bibliothèques du langage C et de l'environnement POSIX.

La bibliothèque **stdio.h** est utilisée pour les opérations d'entrée-sortie, notamment l'ouverture, la lecture et la fermeture des fichiers.

La bibliothèque **stdlib.h** est utilisée pour les conversions numériques, l'allocation dynamique et la gestion générale des ressources.

La bibliothèque **string.h** permet de manipuler les chaînes de caractères, ce qui est indispensable pour comparer les options, traiter les noms de processus et formater les commandes.

La bibliothèque **ctype.h** est utilisée pour tester les caractères, notamment pour vérifier si une chaîne est composée uniquement de chiffres.

La bibliothèque **dirent.h** permet de parcourir les répertoires. Elle est essentielle pour ouvrir /proc et lire ses entrées.

La bibliothèque **unistd.h** fournit des fonctions POSIX comme getuid et sysconf.

La bibliothèque **pwd.h** permet de convertir un UID numérique en nom d'utilisateur lisible.

La bibliothèque **grp.h** peut être utilisée pour manipuler les informations liées aux groupes.

Les bibliothèques **sys/types.h** et **sys/stat.h** fournissent des types et structures utiles à la programmation système.

COMPILATION ET EXECUTION

Pour compiler le programme, utilisez le compilateur gcc avec la commande **gcc myps.c -o myps**, puis lancez l'exécutable obtenu via la commande **./mysp**. Ce programme s'exécute soit sans option pour un affichage simple, soit avec différentes options pour adapter son comportement selon vos besoins, telles que :

- **./mysp -a,**
- **./mysp -e,**
- **./mysp -u,**
- **./mysp -f,**
- **./mysp -p 1,**
- **./mysp -C bash,**
- **./mysp -G 1000,**
- **./mysp -L,**
- **./mysp -H,**
- **./mysp -Z,**
- **--sort=pid,**
- **--sort=cpu**
- **--sort=nom.**

TESTS ET VALIDATION

La validation du projet consiste à vérifier que les fonctionnalités attendues sont bien présentes et produisent des résultats cohérents. La méthode de test consiste à exécuter la commande **mysp** avec différentes options, puis à comparer les résultats avec ceux de la commande **ps** lorsque cela est possible.

Tableau récapitulatif des tests

Fonctionnalité testée	Commande utilisée	Résultat attendu	Résultat obtenu	Statut
Affichage par défaut	./mysps	Affichage simple des processus	Conforme	Validé
Filtrage par PID	./mysps -p 1	Un seul PID affiché	Conforme	Validé
Affichage détaillé	./mysps -f	Colonnes supplémentaires	Conforme	Validé
Filtrage par nom	./mysps -C bash	Processus bash affichés	Conforme	Validé
Threads	./mysps -L	Affichage des TID	Conforme	Validé
Hierarchie	./mysps -H	Arbre des processus	Conforme	Validé
Tri	./mysps -- sort=pid	Tri par PID	Conforme	Validé

ORGANISATION COLLABORATIVE DU PROJET

Le projet a été réalisé en groupe. Afin de faciliter le développement, les fonctionnalités ont été réparties entre les différents membres. Chaque membre s'est vu attribuer un ensemble d'options à implémenter ou à tester.

Cette organisation a permis d'avancer plus rapidement, car plusieurs parties du programme pouvaient être développées en parallèle. Elle a également permis à chaque membre de se concentrer sur une catégorie précise de fonctionnalités.

L'intégration finale a consisté à regrouper les différentes contributions dans une version commune du programme. Cette étape a demandé une harmonisation du code, notamment

au niveau des noms de variables, de la structure utilisée pour représenter les processus, de la gestion des options et du format d’affichage.

Lorsque des conflits ou incohérences apparaissaient, ils étaient corrigés en vérifiant le comportement attendu de la commande et en testant à nouveau les options concernées.

DIFFICULTES RENCONTREES ET SOLUTIONS APPORTEES

La première difficulté rencontrée a été la compréhension du fonctionnement de `/proc`. Contrairement à un fichier classique, `/proc` est un système de fichiers virtuel dont le contenu change en temps réel. Il a donc fallu comprendre quels fichiers lire et quelles informations exploiter.

La deuxième difficulté concernait le fichier `/proc/[pid]/stat`. Ce fichier contient de nombreux champs sur une seule ligne, ce qui rend son analyse délicate. Pour résoudre ce problème, nous avons identifié uniquement les champs nécessaires au projet et utilisé une lecture structurée pour les extraire.

La troisième difficulté était liée à la disparition possible des processus pendant l’exécution du programme. Un processus peut être présent lors du parcours de `/proc`, puis disparaître avant la lecture de ses fichiers. Pour gérer ce cas, le programme vérifie systématiquement si les fichiers peuvent être ouverts et ignore proprement les processus devenus inaccessibles.

Une autre difficulté concernait la lecture du fichier `cmdline`. Les arguments y sont séparés par des caractères nuls et non par des espaces. Il a donc fallu transformer ces caractères pour obtenir une ligne de commande lisible.

L’affichage des threads a également constitué une partie plus avancée. Il a fallu comprendre que les threads d’un processus sont accessibles dans le dossier `/proc/[pid]/task`.

Enfin, l’intégration des différentes fonctionnalités a demandé une attention particulière. Chaque option devait fonctionner seule, mais aussi en combinaison avec d’autres options. Des tests successifs ont donc été nécessaires pour stabiliser le programme.

LIMITES ACTUELLES DU PROGRAMME

Même si le programme myps est fonctionnel, il présente certaines limites. Tout d’abord, il ne reproduit pas toute la complexité de la commande ps originale. La commande ps officielle possède un très grand nombre d’options, avec plusieurs styles d’arguments hérités d’Unix et de BSD. Notre projet se limite à un ensemble d’options sélectionnées.

Ensuite, certains calculs, comme le pourcentage CPU, peuvent être approximatifs. La commande ps originale utilise des méthodes plus complètes et optimisées pour présenter certaines informations.

Le programme dépend également du système de fichiers `/proc`. Il est donc spécifique à Linux et ne peut pas fonctionner tel quel sur un autre système d’exploitation qui ne possède pas cette interface.

L’affichage peut aussi être limité lorsque les lignes de commande sont très longues. Pour conserver une sortie lisible, le programme peut tronquer certaines commandes.

Enfin, certaines informations comme le contexte SELinux ne sont disponibles que sur les systèmes où SELinux est activé. Sur d'autres systèmes, cette colonne peut contenir une valeur par défaut.

PERSPECTIVES D'AMELIORATION

Nous envisageons plusieurs améliorations pour faire évoluer le programme. Une première amélioration serait d'ajouter davantage d'options de la vraie commande ps, notamment les options avancées liées à la priorité, au temps de démarrage, à la mémoire virtuelle ou aux colonnes personnalisées.

Une deuxième amélioration serait de rendre le programme plus modulaire en séparant le code en plusieurs fichiers. Par exemple, on pourrait créer un fichier pour la lecture de /proc, un fichier pour le filtrage, un fichier pour l'affichage, un fichier pour le tri et un fichier d'en-tête contenant les structures et prototypes.

La troisième amélioration serait d'ajouter un Makefile afin de faciliter la compilation du projet.

La quatrième amélioration consisterait à améliorer le calcul du pourcentage CPU et à ajouter le calcul du pourcentage mémoire.

La cinquième amélioration par contre serait de produire un mode d'affichage interactif, proche de la commande top, avec une actualisation périodique des processus.

Et enfin, il serait possible d'ajouter des tests automatisés afin de vérifier rapidement que chaque option fonctionne après modification du code.

CONCLUSION GENERALE

La réalisation de ce projet nous a permis de mieux comprendre le fonctionnement interne de la commande `ps` et, plus largement, la manière dont Linux expose les informations relatives aux processus. À travers le développement de `myps`, nous avons appris à parcourir le système de fichiers virtuel `/proc`, à lire des fichiers système, à extraire des informations sur les processus, à appliquer des filtres, à trier des données et à produire un affichage lisible pour l'utilisateur. Ce projet nous a également permis de mettre en pratique plusieurs notions fondamentales du cours de Théorie des Systèmes d'Exploitation, notamment les processus, les threads, les PID, les PPID, les UID, les GID, les sessions, les groupes de processus et la séparation entre espace utilisateur et espace noyau. Sur le plan de la programmation, ce travail a renforcé notre maîtrise du langage C, en particulier la manipulation des fichiers, des répertoires, des chaînes de caractères, des structures, de l'allocation dynamique et de la gestion des erreurs. En définitive, ce projet constitue une application concrète de la programmation système sous Linux. Il montre comment un outil utilisateur peut interroger les informations maintenues par le noyau et les présenter sous une forme exploitable.